



NAME : HAMDAN ALI

WEEK 5 : Day 23

DAY 23

Task : Screenshoots Of Output

CODEBASE Of the Work

```
Week5 > Day24 > src > components > FormField.jsx > ...
2
3 const FormField = ({
4   label,
5   name,
6   type = 'text',
7   value,
8   onChange,
9   error,
10  required,
11  placeholder,
12  maxLength
13 }) => {
14   const isTextarea = type === 'textarea';
15   const InputComponent = isTextarea ? 'textarea' : 'input';
16
17   // Validation icon logic (simple version)
18   // For required fields, if they have value and no error, show success
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

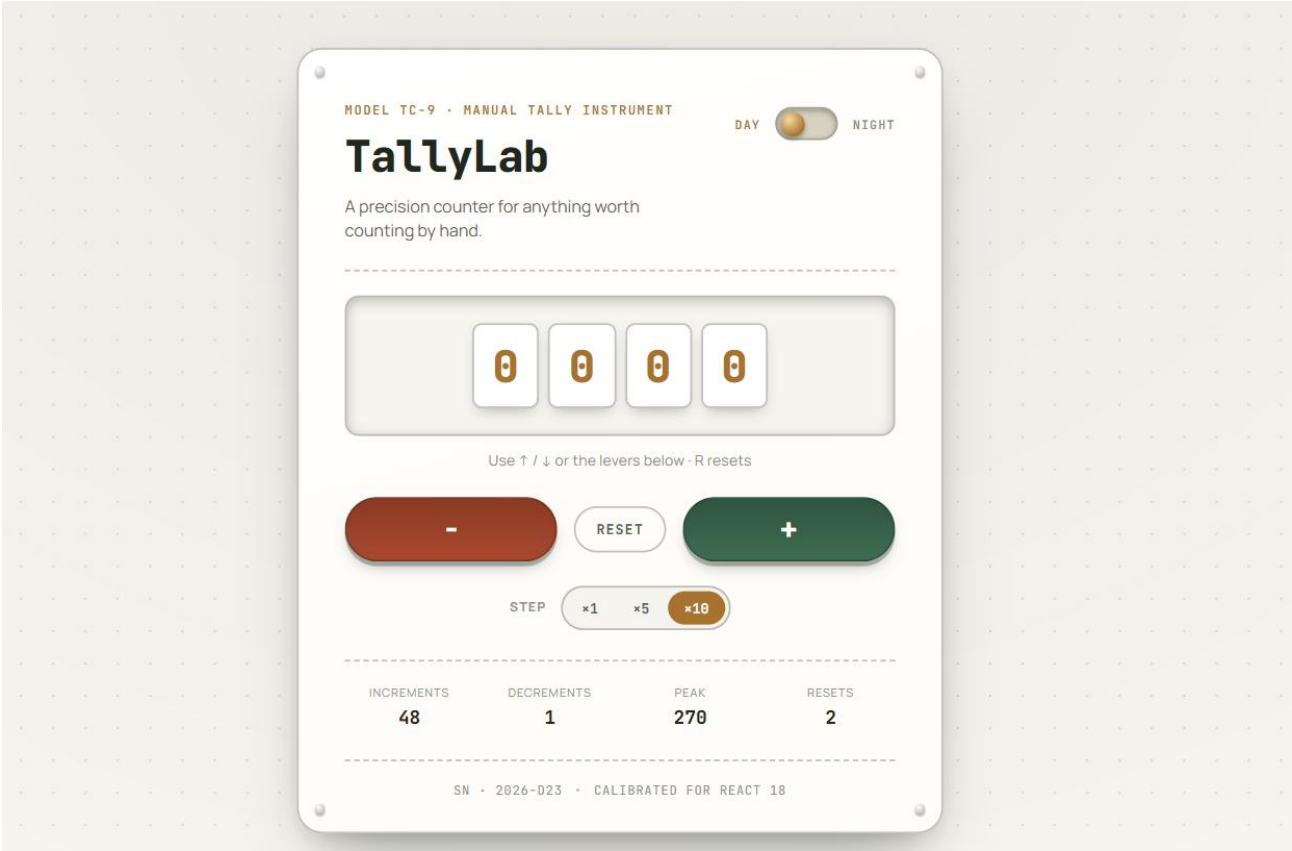
```
PS D:\tasks\Hamdan(FullStack_Internship_At_HindukushSoft Technologies)\internship (HindukushSoft Technologies)\Week5\
  run "npm fund" for details

found 0 vulnerabilities
PS D:\tasks\Hamdan(FullStack_Internship_At_HindukushSoft Technologies)\internship (HindukushSoft Technologies)\Week5\
  > day24-form-live-preview@1.0.0 dev
  > vite

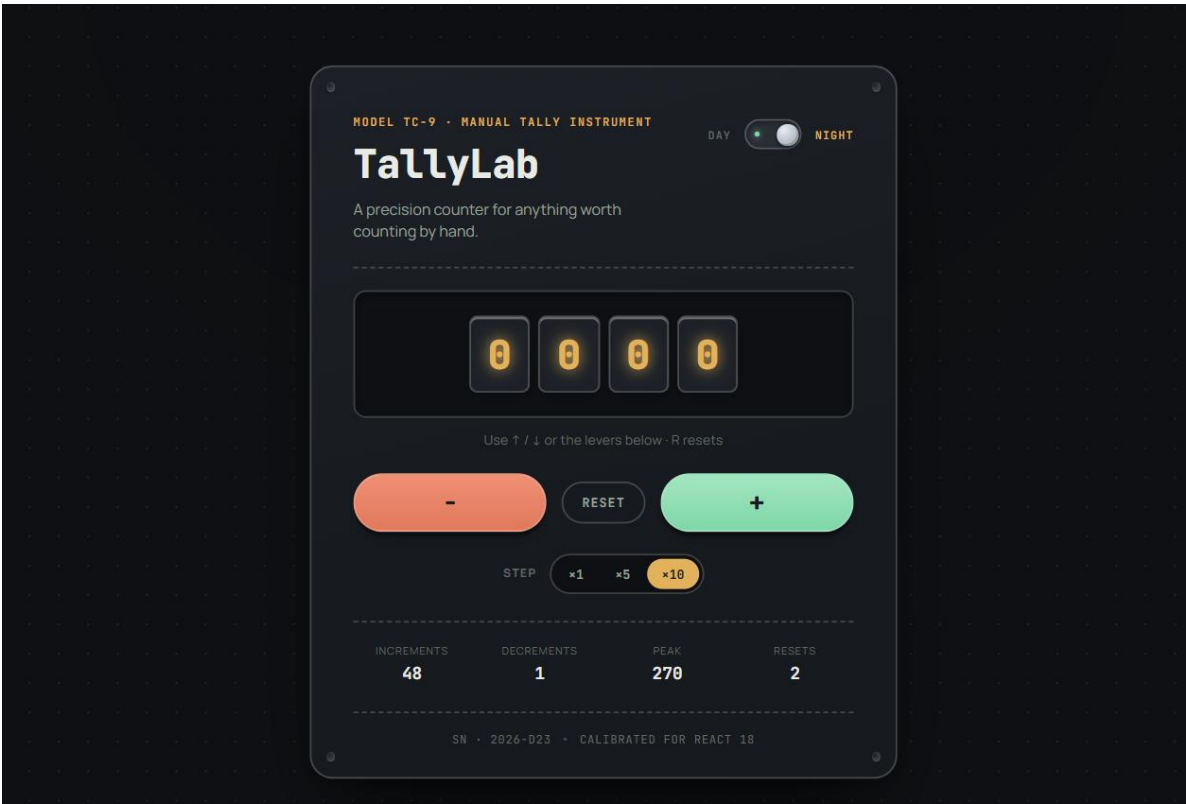
1:59:45 PM [vite] warning: `esbuild` option was specified by "vite:react-babel" plugin. This option is deprecated, please
`optimizeDeps.rollupOptions` / `ssr.optimizeDeps.rollupOptions` is deprecated. Use `optimizeDeps.rollupOptions` instead
plugin. Set VITE_DEPRECATION_TRACE=1 to see where it is called.
[vite:react-babel] We recommend switching to `@vitejs/plugin-react-oxc` for improved performance. More information at https://vitejs.dev/guide/optimizations.html#react-oxc
Warning: Invalid input options (1 issue found)
- For the "jsx". Invalid key: Expected never but received "jsx".

VITE v8.2.1 ready in 539 ms

→ Local: http://localhost:5173/
```

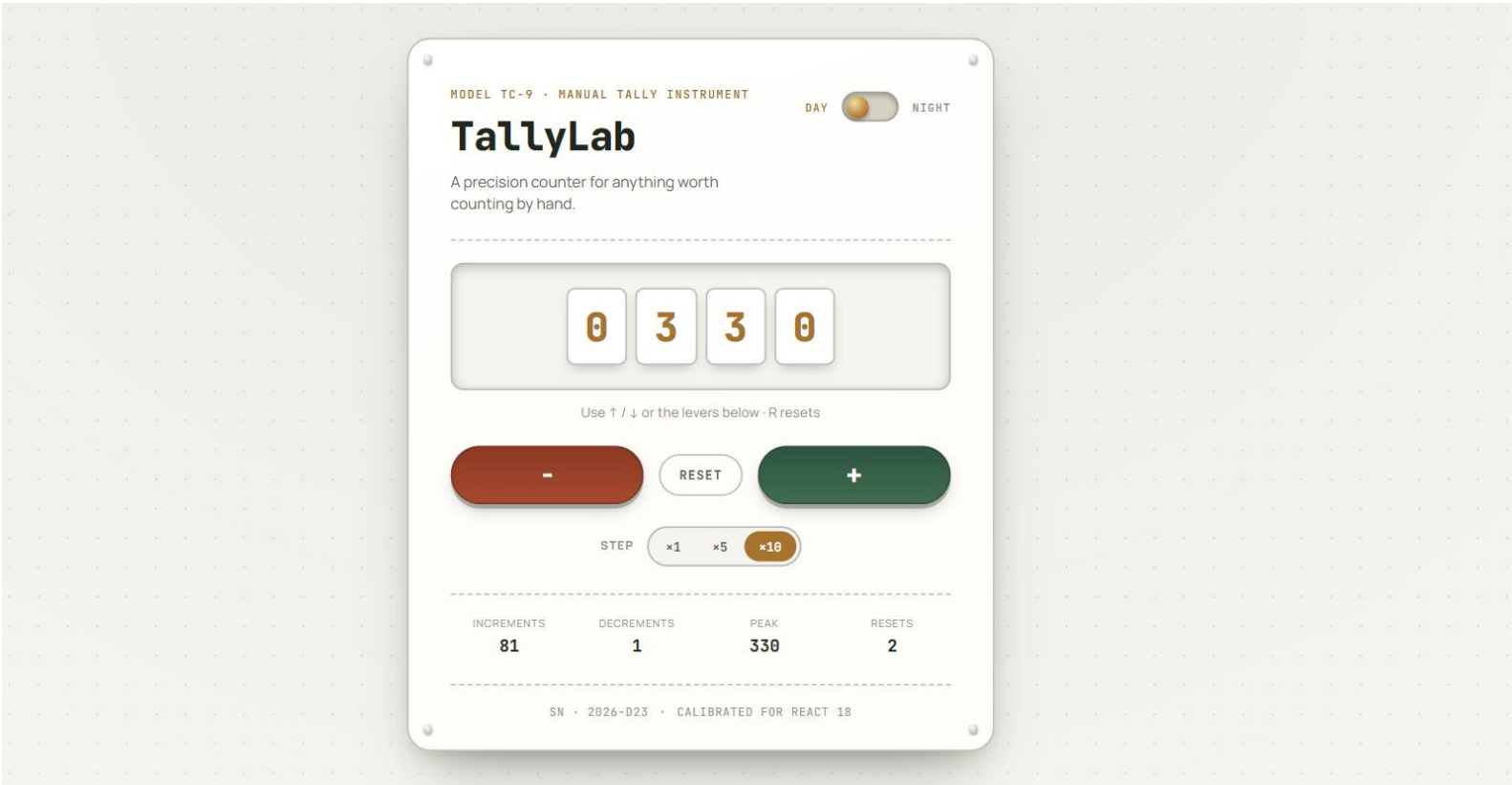


Dark theme:





After Doing Changes:



EXPLANATION OF THE PROJECT

DAY 24

Week 5 · Day 23

Counter App and Theme Toggle with useState

What today was about

Today's task was to get comfortable with two things that end up underneath almost every interactive screen in React: the useState hook, and the event handlers that feed it. On paper the brief was simple — build a counter and a light/dark toggle. In practice I treated it as a chance to actually sit with how state updates flow through a component, how stale closures sneak into keyboard handlers, and how a UI can be given a bit of personality instead of shipping the first layout that comes to mind.

Rather than building the usual grey card with a number and two buttons, I framed the exercise around a real object: a hand-tally counter, the kind used at gate counts, in labs, or on library desks to keep a running total by hand. That gave me a concrete vocabulary to design against — a sunken digit readout, click levers instead of flat buttons, and a rocker light switch for the theme toggle, styled like the panel switch on a piece of lab equipment rather than a generic pill toggle. I called the finished piece TallyLab.

Design approach

Before touching any code I sketched out what the panel needed to communicate: a readout you trust at a glance, controls that feel like they have weight, and a lighting change that feels like flipping a switch on real hardware rather than a colour swap. That's where the two theme names came from — Day Lab and Night Lab — instead of the usual 'light mode / dark mode' labelling.

Palette and type

The palette stays close to instrument-panel materials: a porcelain and brass combination for Day Lab, and a near-black panel with a warmer brass and a soft phosphor-green for Night Lab, which gives the digits a faint glow the way an old LED display would. Numbers are set in JetBrains Mono, because a counter reading in a proportional font never looks quite right — the digits need to line up like they would on a real



wheel. Labels and body copy use Manrope, a plainer humanist sans, so the mono face stays reserved for numbers and doesn't get diluted.

The signature detail

The one piece of the interface I spent the most time on is the digit readout. Each digit sits inside its own little sunken window, and when a digit changes it slides in and settles, the way a mechanical wheel flips over to its next number. It's a small thing, but it's the difference between 'a number in a box' and something that feels like it's actually being tallied.

Project structure

The project is a small, standard Vite + React app so it can be opened, run, and extended without any extra setup:

```
Day23/
├── index.html
├── package.json
├── vite.config.js
├── README.md
├── src/
│   ├── main.jsx
│   ├── App.jsx      // owns the theme state, lays out the panel
│   ├── App.css
│   ├── index.css    // colour tokens for Day Lab / Night Lab
│   └── components/
│       ├── Counter.jsx // the tally mechanism itself
│       ├── Counter.css
│       ├── ThemeToggle.jsx
│       └── ThemeToggle.css
```

I kept the two features in their own components on purpose — App.jsx doesn't know how the counter increments, and Counter.jsx doesn't know anything about the theme. Each piece of state lives exactly where it's used, which made both pieces easy to reason about on their own.

How the useState hook is used

There are three separate calls to useState across the app, and each one is doing a genuinely different job, which was the point of the exercise for me — not just 'store a number', but see the range of things a single hook ends up covering.

1. Theme state — App.jsx

The App component owns one state value, theme, which is either "day" or "night". Instead of always starting on a hard-coded default, the initial value is worked out lazily: it checks localStorage for anything saved from a previous visit, and if there's nothing there yet, it falls back to the visitor's OS-level colour scheme preference. That's the useState lazy-initialiser pattern — passing a function to useState instead of a plain value, so the lookup only runs once on mount rather than on every re-render.

```
const [theme, setTheme] = useState(() => {
  const saved = window.localStorage.getItem(THEME_STORAGE_KEY);
  if (saved === "day" || saved === "night") return saved;
  return prefersNight ? "night" : "day";
});
```

A useEffect watches theme and, whenever it changes, writes the new value onto the <html> element as a data-theme attribute (which is what the CSS keys off) and persists it back to localStorage.

2. Counter value, step size and stats — Counter.jsx

Counter.jsx holds four separate pieces of state: the tally itself (count), the step size the buttons move by (step — 1, 5 or 10), a small stats object tracking increments, decrements, resets and the session's peak value, and a limitHit flag used only to trigger the warning message and shake animation when the counter hits its floor or ceiling.

count is restored from localStorage the same lazy way the theme is, clamped to a sensible range on load so a corrupted or stale value can't leave the display in a broken state. Every time count changes, a small effect re-saves it, so refreshing the page — or coming back tomorrow — picks up right where the tally left off.

3. A tiny local state inside ThemeToggle

The switch itself also holds one small piece of local state, isPressed, purely so it can show a pressed-down look while the mouse or a finger is held on it. This one never leaves the component and never needs to — a good reminder that not every piece of state needs to live at the top of the tree; it should live wherever it's actually used.

How events are handled

The second half of the brief was events, and I tried to cover a genuinely useful spread rather than just onClick everywhere.

Feature	What it does
onClick	Drives the increment, decrement and reset levers, and the step-size options ($\times 1$ / $\times 5$ / $\times 10$).
onMouseDown / onMouseUp / onMouseLeave	Used on the theme switch to show a pressed state while it's actually being held, and to release cleanly if the pointer drags off the control.
onTouchStart / onTouchEnd	Mirrors the pressed state on touch screens, so the switch feels consistent on mobile.
onKeyDown (on the switch)	Lets the theme switch respond to Space or Enter, since it's a custom control rather than a native checkbox.
window keydown listener (useEffect)	Registered once in Counter.jsx so the whole page can be driven from the keyboard: Arrow Up increments, Arrow Down decrements, and R resets — skipped automatically if focus is inside a text field.

The keyboard listener is the one place I had to be careful about a classic React gotcha: closures going stale. The handler reads count and step to decide what the next value should be, so the effect that registers the listener has to re-run whenever either of those changes — otherwise pressing the arrow keys quickly would keep incrementing from an old, cached count instead of the current one. Re-registering the listener on every relevant change keeps it reading live values without needing refs to work around it.

There's a second, quieter React lesson buried in the increment and decrement handlers. My first pass wrote them as functional state updates (`setCount(current => ...)`) and fired a second `setState` call for the stats from inside that updater. That works, but in development React's Strict Mode intentionally invokes updater functions twice to help surface side effects like this, which was quietly double-counting the stats. Reading count directly instead of through a functional updater fixed it, and was the right call anyway, since these handlers only ever run in response to a single, discrete user action — not the kind of rapid overlapping updates functional updaters are meant to protect against.

Feature summary

Feature	What it does
Digit-reel readout	A four-digit mechanical-style display, zero-padded, that flips each digit into place as it changes.
Range clamping	The count is bounded between 0 and 9,999; hitting either edge triggers a brief shake and an inline warning instead of silently doing nothing.

Feature	What it does
Step selector	Switch between moving the tally by 1, 5 or 10 per press.
Session stats	Live totals for increments, decrements, resets and the highest value reached this session.
Keyboard control	Arrow Up / Arrow Down / R drive the counter without touching the mouse.
Persistent state	Both the tally and the chosen theme are remembered across page reloads via localStorage.
Day Lab / Night Lab toggle	A rocker-style switch that recolours the whole panel, including a subtle glow on the digits at night.
Accessibility	Every control is reachable by keyboard, focus is always visible, the switch uses role="switch" with aria-checked, and animations respect prefers-reduced-motion.

Styling approach

All theming runs through CSS custom properties defined once in index.css, split between a :root block for Day Lab and a [data-theme="night"] override for Night Lab. Every component references the same variable names — --panel-bg, --ink, --brass, and so on — rather than hard-coded colours, so flipping the theme is a single attribute change on the <html> element and every surface, border and shadow transitions together. That also means there was exactly one place to tune the palette while designing, instead of hunting through every component file.

Keeping colour, spacing and type values as named tokens rather than one-off numbers made the whole panel feel like it was designed as one object, not three separate components bolted together.

Running the project

The app is a standard Vite project, so the usual three commands cover everything:

```
npm install
npm run dev    # local development server
npm run build  # production build, output to dist/
npm run preview # preview the production build locally
```



What I'd build on next

- Add a small history log so each press of a lever is timestamped, not just tallied — useful if this were ever used for real counting sessions.
- Let the range (0–9,999) and the step options be configured from the panel itself, rather than being fixed constants.
- Add a second, independent counter channel — mirroring how physical lab tally counters often track two categories side by side.
- Swap localStorage for a small backend once this internship reaches the full-stack weeks, so a tally could be shared across devices.

Closing thoughts

What stood out to me most from today wasn't the counter logic itself — that part is genuinely simple — but how much a UI's personality changes the way it's used to test. Because the panel looked like a real instrument, I found myself pressing the levers to see how they'd respond, checking the shake animation actually felt like a limit being hit, and tuning the digit-flip timing until it read as mechanical rather than just 'a CSS transition'. That attention is really the whole point of practising useState and events on a small project like this before applying the same patterns to something larger.

THANK YOU!